LLM Wiki 深度技术解读

从「检索增强」到「编译式知识库」的范式跃迁 ——
三份 Gist 构建的完整思想链条，跨越编译器、记忆引擎与分段架构三层设计空间

V1 · Karpathy 基础范式　V2 · rohitg00 生产扩展　V3 · ahumanft 分段架构

目录

1. 总体架构概览
2. V1 · Karpathy 基础范式
3. V2 · rohitg00 生产扩展
4. V3 · ahumanft 分段架构
5. 综合技术分析
6. 参考资料与社区生态

1 总体架构概览

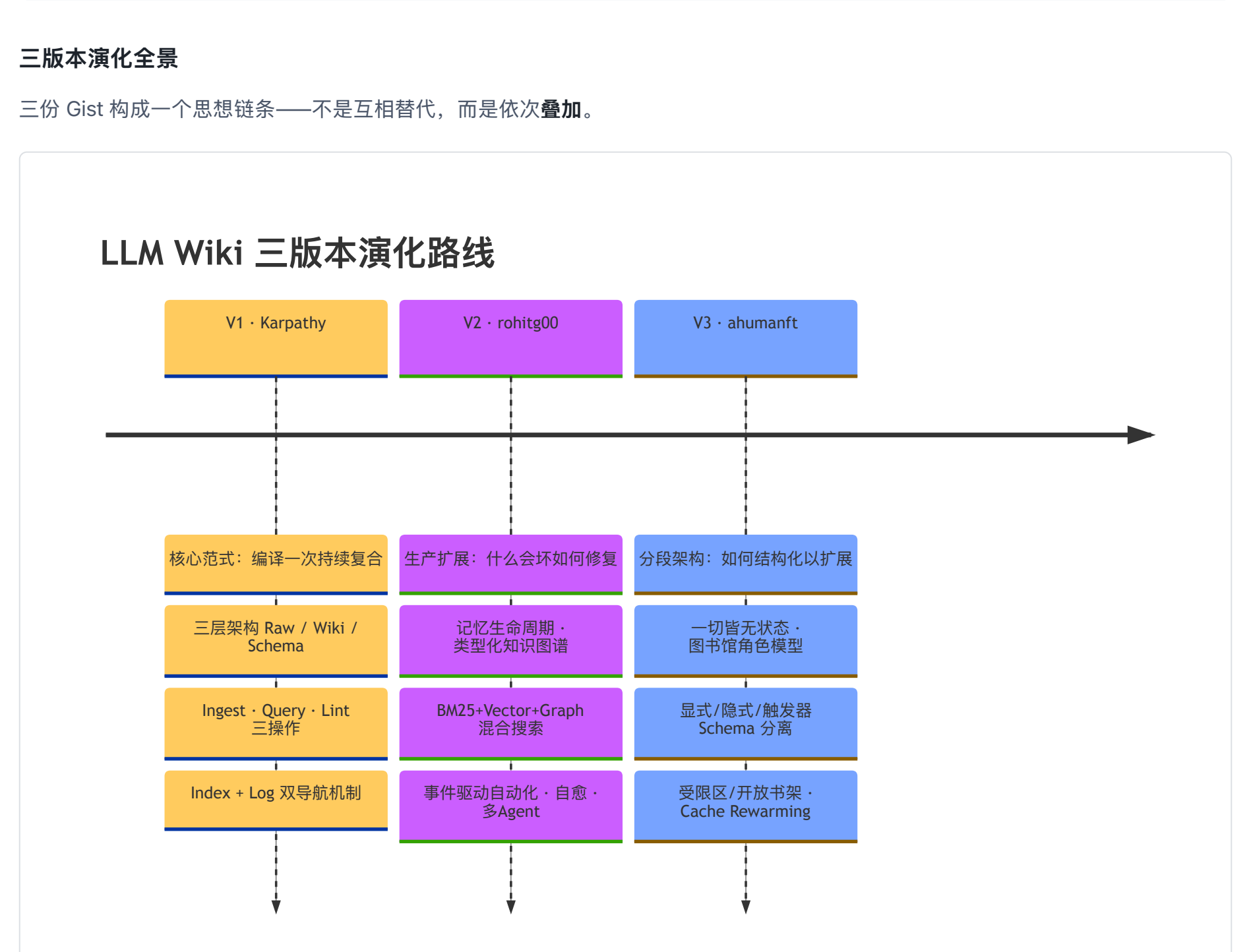
什么是 LLM Wiki?

LLM Wiki 是一个**由大语言模型持续维护和编译的结构化知识库**。它与传统 RAG 的本质区别：RAG 在每次查询时从原始文档「重新发现」知识，而 LLM Wiki 将知识**编译一次、持续复合**。Wiki 本身是**持久化、递增式增长的资产**——跨引用已存在、矛盾已被标记、合成结论已反映所有已读材料。

> **一句话定位：**LLM 是编译器 + 图书管理员；Wiki 是编译产物；人类是主编。这是 Vannevar Bush (1945) Memex 设想的真正落地——不是因为更好的文档或搜索，而是因为终于有了*真正做维护的图书管理员*（LLM）。

三版本演化全景

三份 Gist 构成一个思想链条——不是互相替代，而是依次**叠加**。



LLM Wiki 三版本演化路线
V1 · Karpathy
V2 · rohitg00
V3 · ahumanft
核心范式：编译一次持续复合
三层架构 Raw / Wiki / Schema
Ingest · Query · Lint 三操作
Index + Log 双导航机制
生产扩展：什么会坏如何修复
记忆生命周期 · 类型化知识图谱
BM25+Vector+Graph 混合搜索
事件驱动自动化 · 自愈 · 多Agent
分段架构：如何结构化以扩展
一切皆无状态 · 图书馆角色模型
显式/隐式/触发器 Schema 分离
受限区/开放书架 · Cache Rewarming

图 1-1 · V1 → V2 → V3 演化时间线 —— 每层建立在前层之上

三版本核心贡献对比

维度	V1 KARPATHY	V2 ROHITG00	V3 AHUMANFT
核心隐喻	编译器 (Compiler)	记忆引擎 (Memory Engine)	图书馆 (Library)
架构风格	三层单体 (Raw / Wiki / Schema)	分层记忆 (Working→Episodic→Semantic→Procedural)	角色分段 (Ingestor / Librarian / Linter)
知识表示	Markdown + wikilinks	类型化知识图谱 + 实体提取	段内 Schema 自治，段间松散耦合
搜索策略	index.md 目录 (≤100 页)	BM25 + Vector + Graph + RRF 融合	Librarian 双通道并行路由检索
质量保障	周期性 Lint	置信度 · 取代 · 遗忘 · 自愈	显式/隐式分离 + 触发器精确调用
适用规模	~100 源，个人知识库	数千会话，生产环境	多 Agent 团队，超长会话
待解决问题	不承诺扩展性	单体 Schema 承载所有机制时过载	段间通信与边界治理（开放问题）

2 V1 · Karpathy 的 LLM Wiki 基础范式

2026 年 4 月，Andrej Karpathy 发布了一份「思想文件」(idea file)。它是整个 LLM Wiki 运动的**起源**——不是实现规范，而是一个**可移植模式** (portable pattern)：复制到 Claude Code / OpenAI Codex 中，Agent 协作构建具体实现。

核心洞察：停止重新推导，开始编译

传统 RAG 路径

```
raw files → chunk → embed → vector DB
            ↓
query → retrieve chunks → LLM synthesize
            ↓
      answer discarded — no accumulation
```

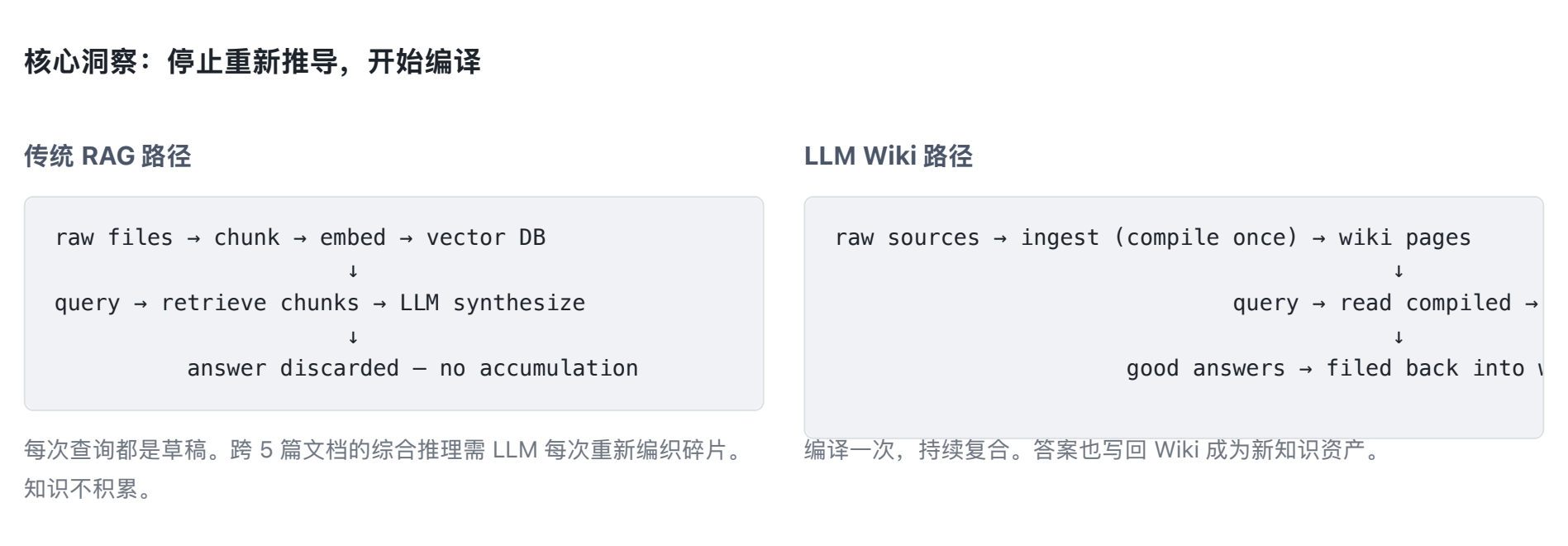
每次查询都是草稿。跨 5 篇文档的综合推理需 LLM 每次重新编织碎片。知识不积累。

LLM Wiki 路径

```
raw sources → ingest (compile once) → wiki pages
                                   ↓
                       query → read compiled →
                                   ↓
               good answers → filed back into
```

编译一次，持续复合。答案也写回 Wiki 成为新知识资产。

三层架构

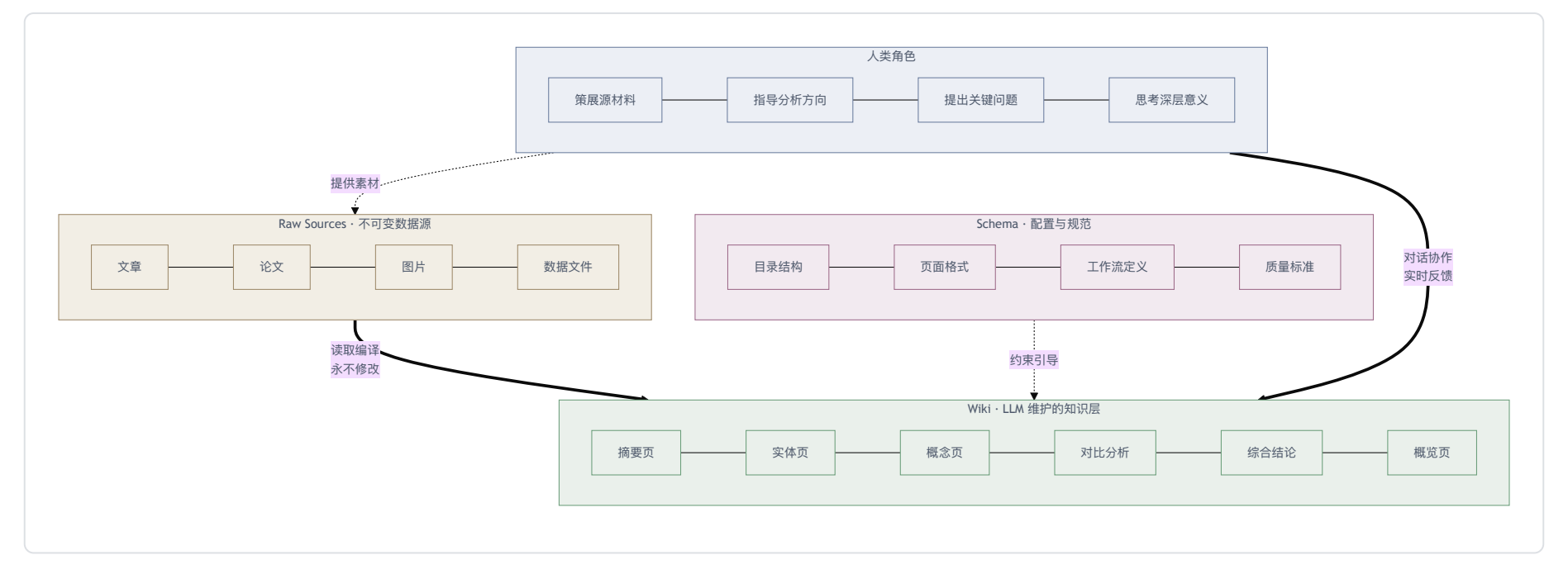


人类角色
策展源材料
指导分析方向
提出关键问题
思考深层意义
提供素材
Raw Sources · 不可变数据源
文章
论文
图片
数据文件
Schema · 配置与规范
目录结构
页面格式
工作流定义
质量标准
对话协作
实时反馈
读取编译
永不修改
约束引导
Wiki · LLM 维护的知识层
摘要页
实体页
概念页
对比分析
综合结论
概览页

图 2-1 · V1 三层架构 —— Raw Sources 不可变是信任的根基

关键原则：**Raw sources 不可变**——LLM 读取但绝不修改。这是数据溯源 (provenance) 的底线。

三个基本操作



Lint · 健康检查
周期触发
扫描矛盾
标记过时声明
检测孤立页
检测缺失概念
建议新方向
Query · 查询管道
接收提问
读取 index 定位
深入相关页面
合成答案+引用
优质答案写回 Wiki
Ingest · 摄入管道
放入 raw/
LLM 读取源
提取关键信息
写入摘要页
更新 index.md
更新实体/概念页
追加 log.md

图 2-2 · Ingest / Query / Lint 三个操作的数据流 —— 一次 Ingest 可触及 10–15 个 Wiki 页面

Index 与 Log 双导航

文件	职责	更新方式	查询范式
`index.md`	内容目录：每页链接 + 一行摘要 + 可选元数据	每次 Ingest 时更新	查询时先读 Index 定位，再深入具体页面
`log.md`	时间线：追加式操作记录	每次操作后追加	`grep "^## \[" log.md | tail -5`

工具生态与哲学渊源

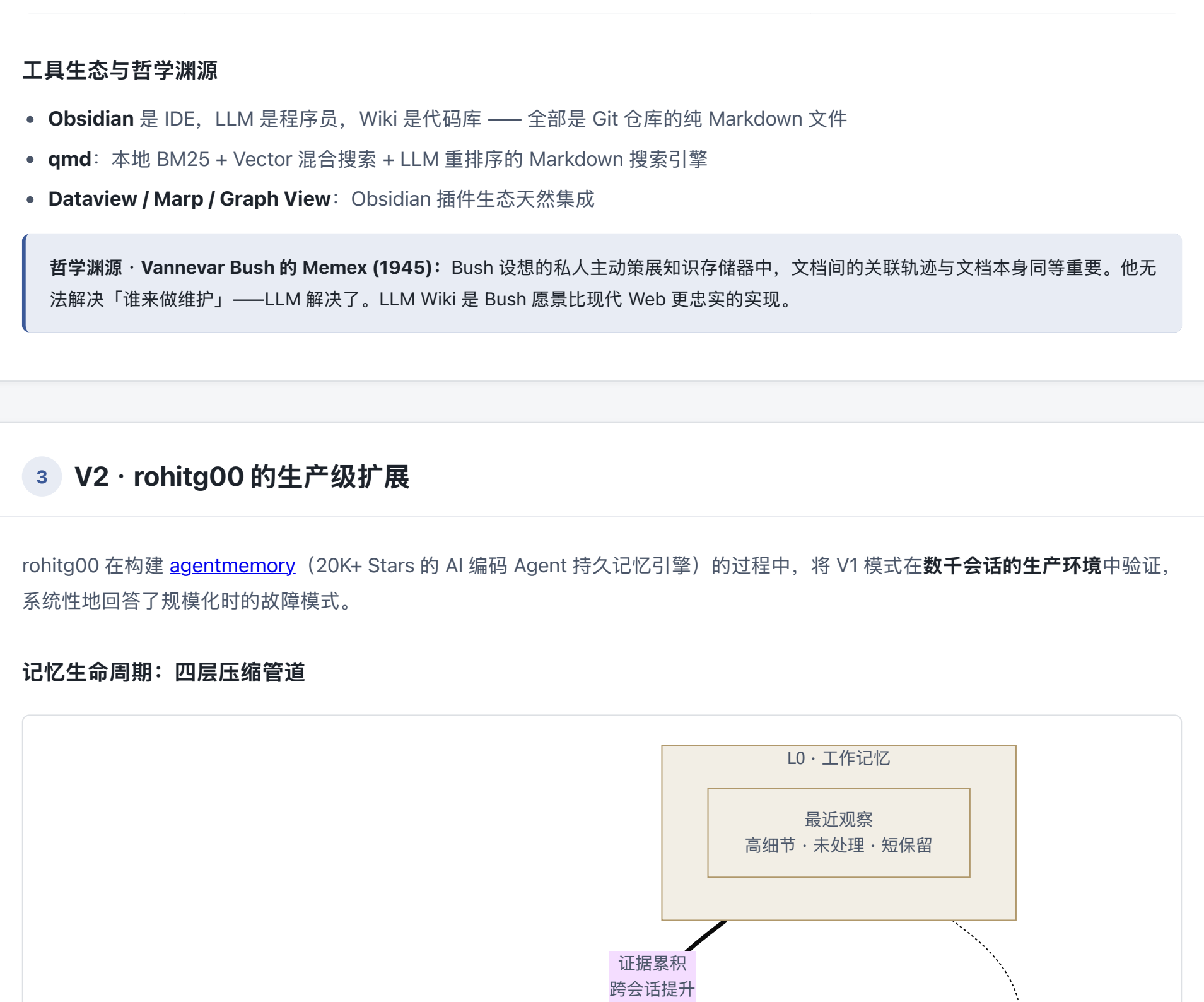
- **Obsidian** 是 IDE，LLM 是程序员，Wiki 是代码库 —— 全部是 Git 仓库的纯 Markdown 文件
- **qmd**：本地 BM25 + Vector 混合搜索 + LLM 重排序的 Markdown 搜索引擎
- **Dataview / Marp / Graph View**：Obsidian 插件生态天然集成

> **哲学渊源 · Vannevar Bush 的 Memex (1945)：**Bush 设想的私人主动策展知识存储器中，文档间的关联轨迹与文档本身同等重要。他无法解决「谁来做维护」——LLM 解决了。LLM Wiki 是 Bush 愿景比现代 Web 更忠实的实现。

3 V2 · rohitg00 的生产级扩展

rohitg00 在构建 agentmemory（20K+ Stars 的 AI 编码 Agent 持久记忆引擎）的过程中，将 V1 模式在**数千会话的生产环境**中验证，系统性地回答了规模化时的故障模式。

记忆生命周期：四层压缩管道



L0 · 工作记忆
最近观察
高细节 · 未处理 · 短保留
证据累积
跨会话提升
L1 · 情景记忆
会话摘要
中等细节 · 压缩自工作记忆
保留: 数天 · 衰减: 快速
多源确认
置信度提升
L2 · 语义记忆
跨会话事实
低细节 · 高置信度 · 多源确认
保留: 数周 · 衰减: 中等
模式提取
归纳固化
L3 · 程序记忆
工作流与模式
最低细节 · 最高置信度 · 长期稳定
保留: 数月-年 · 衰减: 缓慢
保留: 长期 · 衰减: 几乎不

图 3-1 · 四层记忆压缩管道 —— 从「我见过一次」逐步晋升为「事情就是这样工作的」

生命周期三机制

置信度评分

每条声明携带：支持源数量、最后确认时间、是否存在矛盾、数值置信度 (0–1)。随时间衰减，因强化而上升。

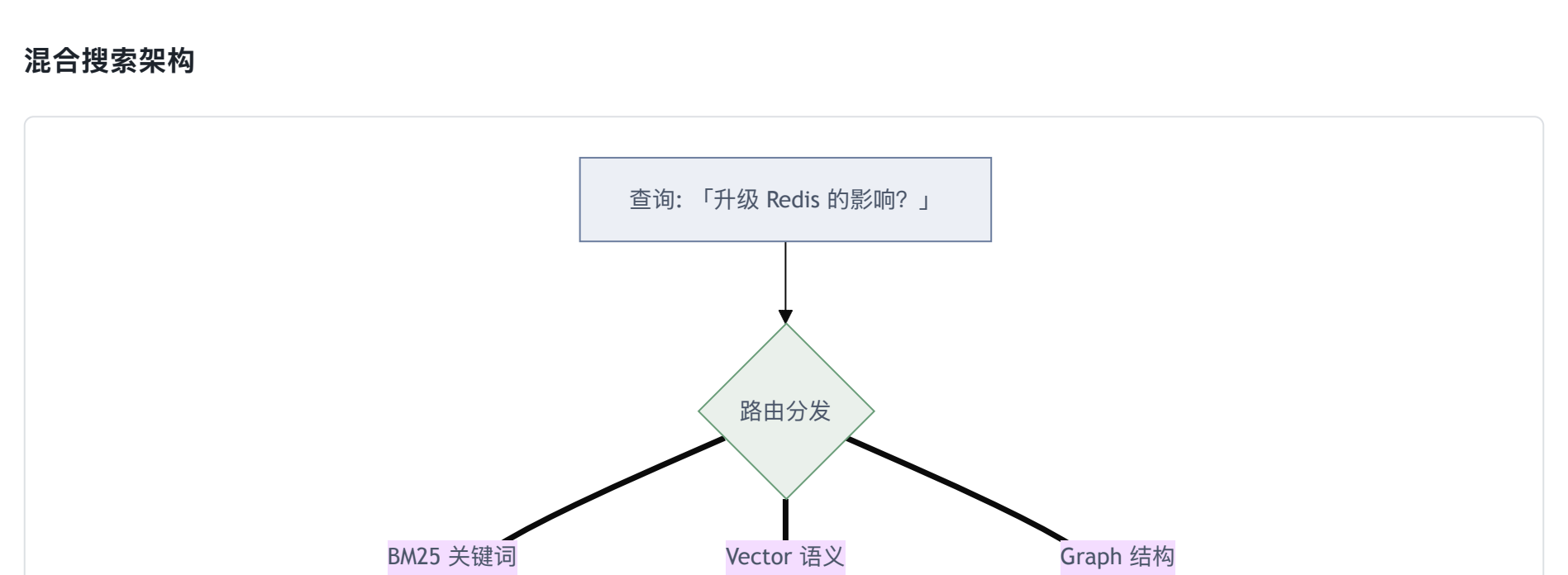
取代

新信息明确**取代**旧声明。链接新旧、时间戳标记、旧版本保留但标记「已过时」。知识的版本控制。

遗忘曲线

借鉴艾宾浩斯：保留率 = $e^{-t/\tau}$。架构决策 τ 长（慢衰减），瞬时 bug τ 短（快衰减）。每次访问重置曲线。

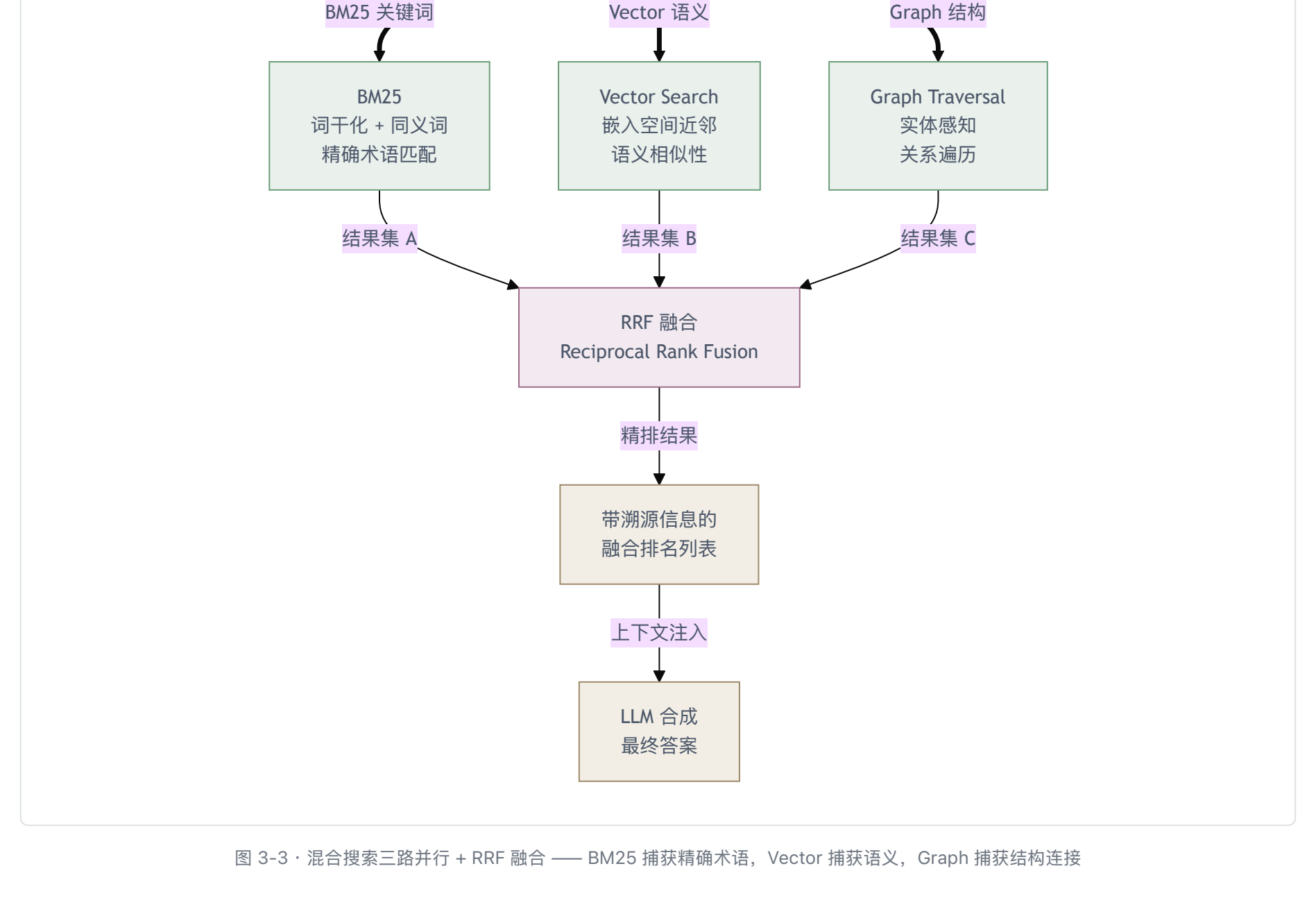
知识图谱：从平面页面到类型化关系



Sarah
Person
owns
负责
Auth Migration
Project
depends_on
依赖
React
Library
depends_on
依赖
redis-config.md
File
depends_on
依赖
mentions
提及
Redis Timeout Bug
Issue
caused_by
根因

图 3-2 · 类型化知识图谱 —— 页面用来读，图谱用来导航和发现。当问到「升级 Redis 的连锁影响？」时，沿 depends_on 边向外遍历

混合搜索架构



查询：「升级 Redis 的影响？」
路由分发
BM25 关键词
Vector 语义
Graph 结构
BM25
词干化 + 同义词
精确术语匹配
Vector Search
嵌入空间近邻
语义相似性
Graph Traversal
实体感知
关系遍历
结果集 A
结果集 B
结果集 C
RRF 融合
Reciprocal Rank Fusion
精排结果
带溯源信息的
融合排名列表
上下文注入
LLM 合成
最终答案

图 3-3 · 混合搜索三路并行 + RRF 融合 —— BM25 捕获精确术语，Vector 捕获语义，Graph 捕获结构连接

事件驱动自动化钩子

事件钩子	触发动作
`On new source`	自动摄入 → 提取实体 → 更新图谱 → 更新索引
`On session start`	从 Wiki 加载基于最近活动的相关上下文
`On session end`	压缩会话为观察，归档洞察
`On query`	评估答案质量；若超过阈值，写回 Wiki
`On memory write`	检查与现有知识的矛盾，触发取代
`On schedule`	周期性 Lint、巩固、保留衰减

质量自愈、结晶化与多 Agent 协同

- **评分一切：**LLM 产出的每条内容带质量评分——结构良好？引用来源？与 Wiki 一致？低于阈值 → 标记或重写。
- **自愈：**Lint 不仅建议——自动修复。孤立页面被链接或标记，过时声明被标记，断裂跨引用被修复。
- **结晶化：**完成的工作链（研究线索、调试会话、分析）被自动蒸馏为结构化文摘。文摘成为一级 Wiki 页面，教训被提取为独立事实。
- **Mesh Sync：**并行 Agent 观察合并至共享 Wiki，Last-write-wins + 时间戳冲突解决。
- **隐私过滤：**摄入前自动剥离 API Key、Token、密码、PII。
- **审计追踪：**每项操作记录时间戳、变更内容、原因——这是问责层。

4 V3 · ahumanft 的分段架构

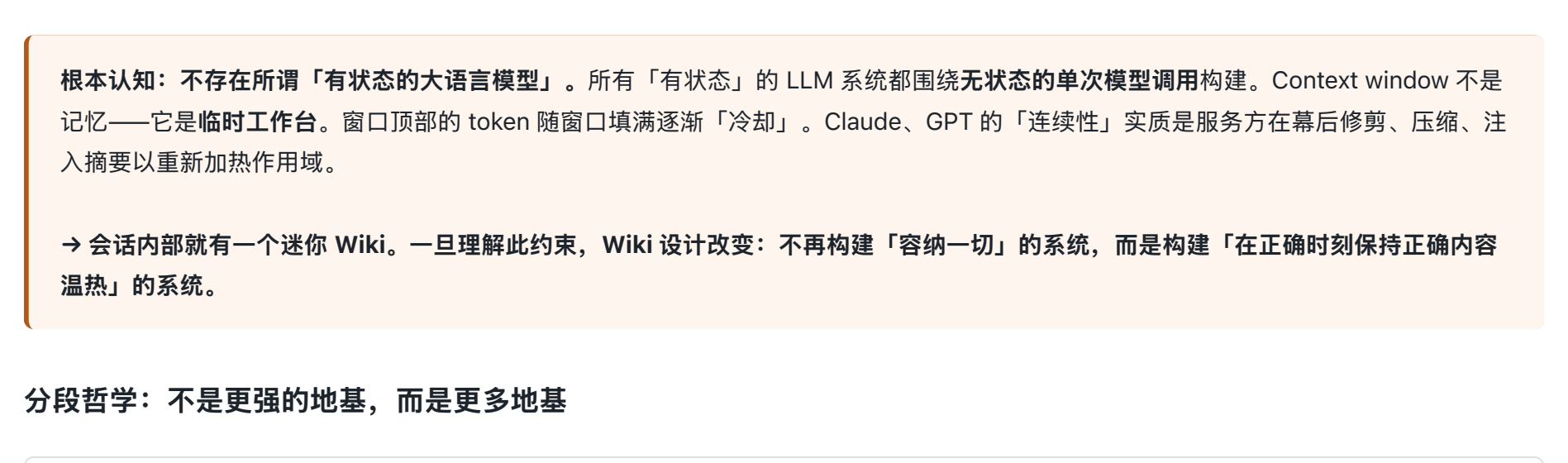
V3 站在 V1（编译器隐喻）和 V2（机制层）之上，提出**分段 (Segmentation) 层**——不是更强的单体，而是构建**多个专用基础**，每个窄到 LLM 在不漂移的前提下可靠执行。

V1 Wiki 即编译器 → V2 编译器做太多时的故障 → V3 构建「没有任何部分需要做太多」的系统

根本约束：一切皆无状态

> **根本认知：不存在所谓「有状态的大语言模型」。**所有「有状态」的 LLM 系统都围绕**无状态的单次模型调用**构建。Context window 不是记忆——它是**临时工作台**。窗口顶部的 token 随窗口填满逐渐「冷却」。Claude、GPT 的「连续性」实质是服务方在幕后修剪、压缩、注入摘要以重新加热作用域。
>
> **→ 会话内部就有一个迷你 Wiki。一旦理解此约束，Wiki 设计改变：不再构建「容纳一切」的系统，而是构建「在正确时刻保持正确内容温热」的系统。**

分段哲学：不是更强的地基，而是更多地基



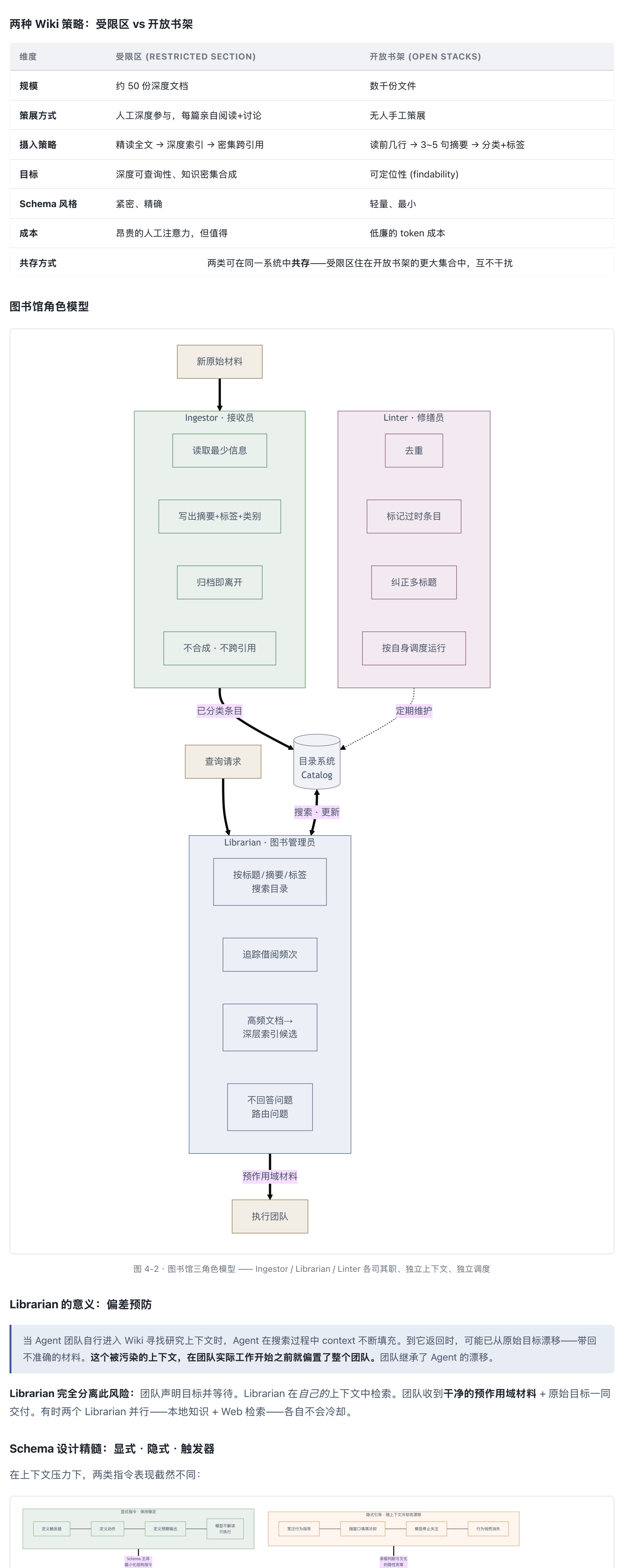
V3 分段架构
受限区 Wiki
深度策展 · 50 文档
开放书架 Wiki
大规模分类 · 数千文件
研究团队 Agent
构建团队 Agent
各段独立 Schema
各段独立 Agent
每段窄到 LLM
可靠执行不漂移
V2 隐含单体
单一 Schema
规则不断累积
单一 Index
条目无限增长
单一 Pipeline
机制层层堆叠
过载
幻觉
漂移
Schema 治理一切
→ 最终破坏一切

图 4-1 · 单体过载 vs 分段架构 —— V2 的故障模式不是数据问题，是分段失败

两种 Wiki 策略：受限区 vs 开放书架

维度	受限区 (RESTRICTED SECTION)	开放书架 (OPEN STACKS)
规模	约 50 份深度文档	数千份文件
策展方式	人工深度参与，每篇亲自阅读+讨论	无人手工策展
摄入策略	精读全文 → 深度索引 → 密集跨引用	读前几行 → 3~5 句摘要 → 分类+标签
目标	深度可查询性、知识密集合成	可定位性 (findability)
Schema 风格	紧密、精确	轻量、最小
成本	昂贵的人工注意力，但值得	低廉的 token 成本
共存方式	两类可在同一系统中**共存**——受限区住在开放书架的更大集合中，互不干扰	

图书馆角色模型



新原始材料
Ingestor · 接收员
读取最少信息
写出摘要+标签+类别
归档即离开
不合成 · 不跨引用
Linter · 修缮员
去重
标记过时条目
纠正多标题
按自身调度运行
已分类条目
定期维护
查询请求
目录系统
Catalog
搜索 · 更新
Librarian · 图书管理员
按标题/摘要/标签
搜索目录
追踪借阅频次
高频文档→
深层索引候选
不回答问题
路由问题
预作用域材料
执行团队

图 4-2 · 图书馆三角色模型 —— Ingestor / Librarian / Linter 各司其职、独立上下文、独立调度

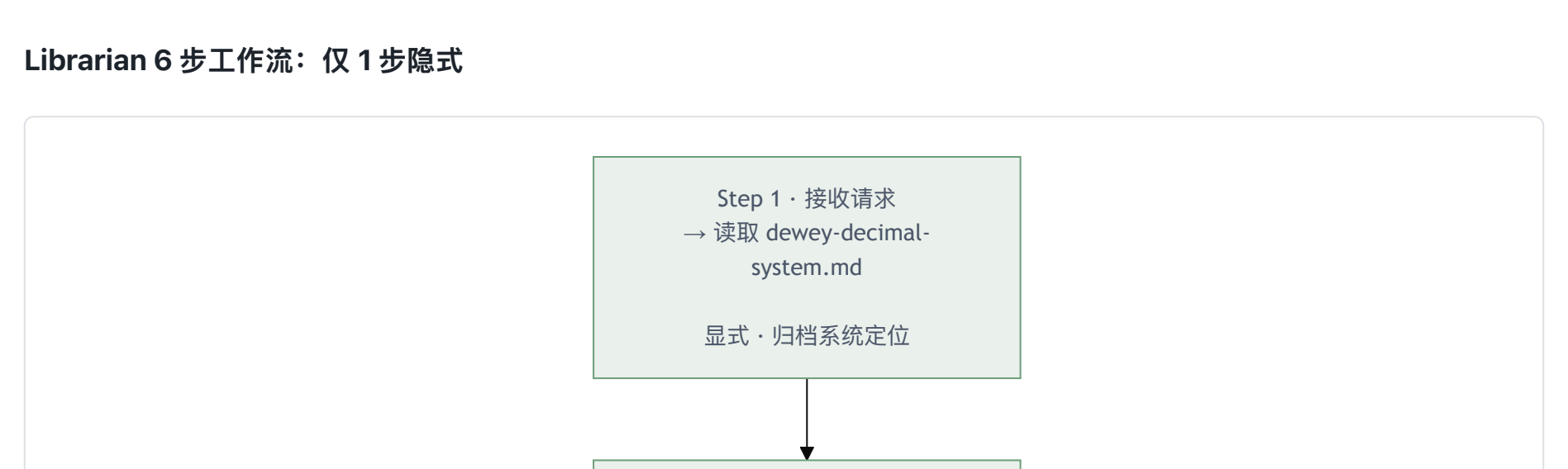
Librarian 的意义：偏差预防

> 当 Agent 团队自行进入 Wiki 寻找研究上下文时，Agent 在搜索过程中 context 不断填充。到它返回时，可能已从原始目标漂移——带回不准确的材料。**这个被污染的上下文，在团队实际工作开始之前就偏置了整个团队。**团队继承了 Agent 的漂移。

Librarian 完全分离此风险：团队声明目标并等待。Librarian 在*自己的*上下文中检索。团队收到**干净的预作用域材料** + 原始目标一同交付。有时两个 Librarian 并行——本地知识 + Web 检索——各自不会冷却。

Schema 设计精髓：显式 · 隐式 · 触发器

在上下文压力下，两类指令表现截然不同：

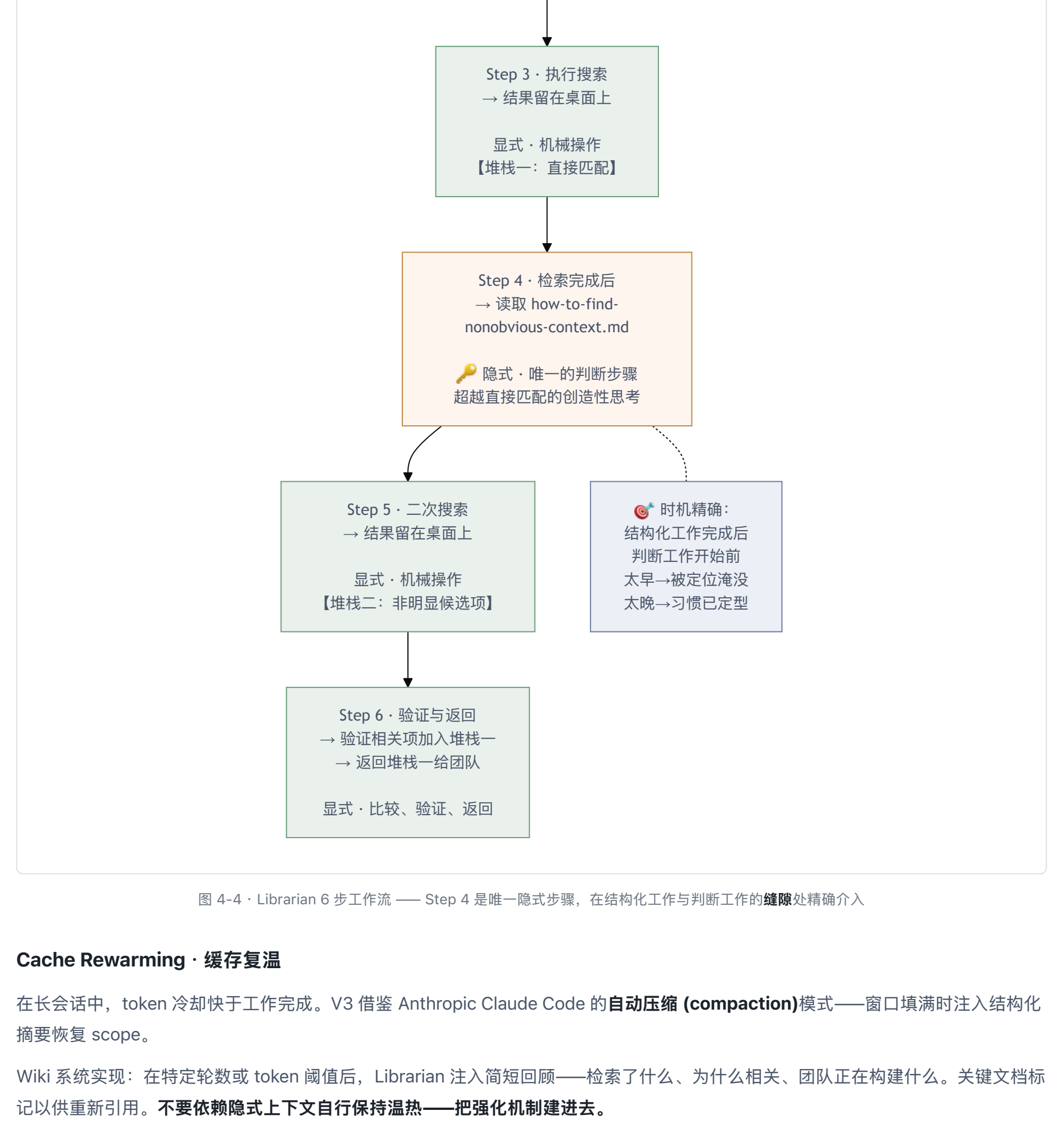


显式指令 · 保持稳定
定义触发器
定义动作
定义预期输出
模型不解读
只执行
Schema 主体
最小化结构指令
Schema
Core
隐式引导 · 随上下文冷却而漂移
宽泛行为指导
随窗口填满冷却
模型停止关注
行为悄然消失
承载判断与文化
的隐性宪章
Implicit
Charters
显式触发器 · 精确时刻激活
按需调用
永不被动加载
在工作流精确
缝隙处介入
在正确时刻
连接两者
工作流
阶段衔接处

图 4-3 · Schema 三层模型 —— 显式指令保持最小化，隐式宪章承载判断，触发器在精确时刻连接

> **核心原则：**显式 Schema 最小化且结构化。隐式宪章承载判断与文化。显式触发器在精确时刻连接。永不被动加载，总是按需调用。触发器时机设计是分段系统中最难的问题——隐式指令在不正确时刻被调用引入方差，模型开始「解读」而非「执行」，Wiki 腐化从此开始。

Librarian 6 步工作流：仅 1 步隐式



Step 1 · 接收请求
→ 读取 dewey-decimal-system.md
显式 · 归档系统定位
Step 2 · 搜索准备
→ 读取 how-to-gather-materials.md
显式 · 定义好的搜索过程
Step 3 · 执行搜索
→ 结果留在桌面上
显式 · 机械操作
【堆栈一：直接匹配】
Step 4 · 检索完成后
→ 读取 how-to-find-nonobvious-context.md
🔑 隐式 · 唯一的判断步骤
超越直接匹配的创造性思考
Step 5 · 二次搜索
→ 结果留在桌面上
显式 · 机械操作
【堆栈二：非明显候选项】
🎯 时机精确：
结构化工作完成后
判断工作开始前
太早→被定位淹没
太晚→习惯已定型
Step 6 · 验证与返回
→ 验证相关项加入堆栈一
→ 返回堆栈一给团队
显式 · 比较、验证、返回

图 4-4 · Librarian 6 步工作流 —— Step 4 是唯一隐式步骤，在结构化工作与判断工作的**缝隙**处精确介入

Cache Rewarming · 缓存复温

在长会话中，token 冷却快于工作完成。V3 借鉴 Anthropic Claude Code 的**自动压缩 (compaction)**模式——窗口填满时注入结构化摘要恢复 scope。

Wiki 系统实现：在特定轮数或 token 阈值后，Librarian 注入简短回顾——检索了什么、为什么相关、团队正在构建什么。关键文档标记以供重新引用。**不要依赖隐式上下文自行保持温热——把强化机制建进去。**

V2 机制在 V3 分段下的正确落地

V2 机制	V3 分段修正
置信度评分	作用域限定到**特定摄入管道**，配合定义的标准才工作
结晶化	当 Librarian 已知哪些文档值得深度索引时才触发
取代	当 Linter 有足够干净的目录来检测重复时才有意义
混合搜索	当 Librarian 在检索前通过正确的过滤器路由查询时才有效
遗忘曲线	不同分段不同 τ 值——受限区几乎不遗忘，开放书架快速轮转

5 综合技术分析

RAG vs LLM Wiki 本质差异

维度	传统 RAG	LLM WIKI
知识积累	无 —— 每次查询从零重新推导	有 —— 编译一次，持续复合
跨引用	需每次重新拼接碎片	已内建于 Wiki 页面中
矛盾处理	无系统性处理	标记 · 取代 · 结晶化
综合结论	不持久化	持久化、可演进
查询效率	嵌入搜索 + LLM 拼装	索引导航 + 图遍历 + 语义搜索
维护成本	低（无维护）	LLM 自动化（近零人工）
认知负荷	全部在查询时发生	分摊到摄入时
信任模型	检索源可见，合成不可溯源	来源引用 + 审计日志 + 置信度

适用规模与场景决策矩阵

场景	推荐版本	关键特征
个人知识库 (<100 源)	V1	Index.md 足够 · Obsidian IDE · 手工工作流
研究项目 (<500 源，多周)	V1 + V2 生命周期	置信度 + 取代 + 结晶化
生产 Agent 系统 (数千会话)	V2	混合搜索 + 知识图谱 + 事件驱动 + 自愈
多 Agent 团队 · 长会话	V2 + V3 分段	Librarian · Cache Rewarming · 显式/隐式分离
企业知识库 (多团队多领域)	V3 全分段	受限区+开放书架 · 共享/私用作用域 · 审计追踪

多学科交叉视角

🧠 认知科学

- Context window = 工作记忆 (容量有限)
- 显式指令 = 程序性知识 (抗干扰)
- 隐式引导 = 陈述性知识 (易褪减)
- 分段 ≈ 认知负荷分段呈现原则
- 遗忘曲线 ← 艾宾浩斯模型

🖥️ 编译器设计

- Ingest = 前端 (parse → IR)
- Wiki pages = 中间表示 (IR)
- Query = 后端 (IR → answer)
- Schema ≈ 编译配置+优化管线
- Lint ≈ 静态分析 pass
- 分段 ≈ 编译单元隔离+LTO

🏛️ 分布式系统

- 分段 = 服务拆分 (Microservices)
- Ingestor/Librarian/Linter = 独立服务
- 段间通信 = 服务间通信 (开放问题)
- 共享/私有作用域 = 多租户隔离
- Mesh Sync = CRDT-like 状态合并
- Cache Rewarming = 热缓存维护

开放问题与未来研究方向

> **V3 核心开放问题：分段本身的配置 (Segmentation Configuration)。**
> 分段在段*内部*工作良好。但段间如何通信？交接如何治理？边界处存在什么规则？——这是 Schema 和导航复杂度快速上升的地方。

1. **段间通信协议：** Librarian 如何知道 Ingestor 刚分类的材料？Push 通知还是 Pull 轮询？中间件模式？
2. **边界 Schema 冲突：** 受限区 Schema 与开放书架 Schema 对同一实体分类冲突时，谁裁决？
3. **跨段知识升级：** 开放书架高频文档何时、如何「晋升」到受限区获得深度索引？
4. **段级资源调度：** 在 token 预算约束下，如何为不同段动态分配上下文窗口配额？类似 OS CPU 调度。
5. **Firsgith 五风险：** 证据作用域 · 认知状态层 · 冲突即研究 · 决策记忆 · 路由器+时间线——可视为 V3 之上的加固层。

6 参考资料与社区生态

原始 Gist

版本	作者	GIST	STARS
V1	Andrej Karpathy	karpathy/llm-wiki	5,000+ ⭐
V2	rohitg00	rohitg00/llm-wiki-v2	1,469 ⭐
V3	ahumanft	ahumanft/LLM-Wiki-V3	5 ⭐

关键衍生项目

项目	作者	焦点
agentmemory	rohitg00	AI 编码 Agent 持久记忆引擎 (20K+ Stars)
okf-harness	pumblus	OKF 兼容的便携式 Agent 维护 Wiki
Loremaester	mm-weber	TRPG 世界构建 (PDF+Markdown → pgvector 混合)
Antinomia	demartinogiuseppe	矛盾作为节点 (而非边) 的 Obsidian 插件
quicky-wiki	silentrob	实体 + 图谱 + BM25 + Vector 四合一实现
gbrain	garrytan	与 V2 设计哲学相似的独立项目
Nemsy (奈姆希)	arrrr110	DeepSeek 超长上下文驱动的个人知识助理
ctx	stevesolun	知识图谱 Agent 工具选择器 (91k+ skills)

相关规范

- Open Knowledge Format (OKF) — GCP 提出的 Agent 记忆知识图谱规范 (v0.1)，以类型化 YAML 前置 Markdown 为基本单元，偏好节点语义而非类型化关系边
- ChristopherA 的命名边实践指南 — `[[wikilinks]]` + named edges (如 `derived_from::[[Source]]`) 的纯文本知识图谱约定

本文基于 spec/links.txt 中链接的三份 Gist 原文深度解读。
分析视角融合认知科学（工作记忆与认知负荷理论）、编译器设计（编译管线与 IR）、分布式系统（服务拆分与状态合并）等多学科框架。
由 Reasonix 生成 · 原始 Gist 分别由 Andrej Karpathy、rohitg00、ahumanft 创作。